CS6868 Research Mini-Project

Segment Queue Synchronizer

Mitesh Sharma
cs25s011@smail.iitm.ac.in

Navaneeth M. Nambiar
ns26z081@smail.iitm.ac.in

Durwasa Chakraborty
cs24d011@smail.iitm.ac.in

April 27, 2026

Abstract

Briefly (150–200 words) describe the problem you tackled, your approach, the headline evaluation results, and the main conclusion. Readers (the graders) should understand the essence of your project from the abstract alone.

1 Goals

State the goals of the project. What concurrent algorithm or data structure did you implement? Why is it interesting? How does it connect to course topics (cite the relevant lecture(s) from the course)?

End this section with a crisp statement of your *research question* (borrowed from or adapted from the chosen project idea in `project_ideas.md`, or formulated by you if you proposed your own topic).

2 Background

Summarise the algorithm or technique at a level that a classmate who has attended the lectures but not read the paper can follow. Cite the primary references [1]. If helpful, include a small figure or pseudo-code listing.

3 Tasks Undertaken

Describe what you actually built. Organise this section as sub-sections per major task; use the *Tasks* list from the project idea as a checklist.

3.1 Implementation

Describe the implementation: key data structures, atomic operations used, subtle invariants, and any design choices you had to make. Link to the relevant files/commits in your GitHub repository.

3.2 Testing and verification

Describe how you tested correctness (QCheck-Lin, QCheck-STM, dscheck, TSAN, etc.) and what bugs the tests caught.

4 Evaluation

Describe your experimental setup (hardware, number of domains/threads, how you measured, how many runs, how you handled warm-up and noise). Present results with plots or tables.

Discuss what the numbers *mean*. Where did the algorithm scale well? Where did it break down? Did the results match your prior expectations?

5 Reflection on the Use of LLMs

Tools and models used. List every LLM-backed tool you used (e.g., GitHub Copilot, ChatGPT, Claude, Gemini, Cursor), the specific model version where relevant (e.g., *GPT-5*, *Claude Sonnet 4.6*, *Gemini 2.5 Pro*), and roughly for what — code generation, debugging, explaining a paper, writing test cases, drafting prose, generating plots, etc.

What worked well. Concrete examples where the LLM genuinely accelerated your work. Did it correctly implement a subtle CAS loop on the first try? Did it explain a paper better than the paper did? Did it catch a race you had missed? Cite specific commits or passages if you can.

What did not work. Concrete examples where the LLM got it wrong or led you astray — wrong memory ordering, subtly broken retry logic, confident but incorrect explanations, hallucinated API calls, plausible-looking citations that did not exist, etc. How did you catch the mistake?

What was surprising. Anything you did not expect — both positive (unexpectedly good at X) and negative (unexpectedly bad at Y). Differences between models or tools?

What was difficult. Parts of the project where the LLM was *unable to substitute for understanding* — reasoning about memory models, debugging a rare interleaving, justifying linearisation points, choosing between design alternatives. What did you end up having to learn yourself?

Your overall assessment. One or two sentences: did the LLM net help or hurt this project? Would you use it the same way again?

6 Conclusions

Answer the research question stated in Section 1. Summarise the main findings. State the limitations of your work honestly. Suggest what you would do next given more time.

Contributions

Member	**Contribution (%)**	**Main responsibilities**
Member One	34%	e.g., lock-free implementation, QCheck-Lin tests
Member Two	33%	e.g., baseline implementations, benchmarking harness
Member Three	33%	e.g., evaluation plots, report, literature survey
Total	**100%**	

References

[1] Firstname Lastname and Other Author. An example paper. *Example Journal*, 1(1):1–10, 2024. doi:10.0000/example.